
Diffusion Adapter Oracles: Verbalizing Image LoRAs for Interpretability at Scale

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We introduce the diffusion adapter oracle (DAO), a meta-model that reads the
2 weights of an image diffusion transformer’s LoRA adapter and says in natural
3 language what that adapter does, without running it. A frozen encoder turns each
4 adapter into tokens at the interpreter’s residual width, the tokens are injected into
5 the interpreter’s residual stream at a fixed layer, and the interpreter is trained to
6 answer questions about the adapter it was given. The interpreter is a language
7 model and the adapters modify a diffusion transformer, so the two share neither
8 an architecture nor a token vocabulary. The interpreter names the concept of a
9 held-out adapter, one minted at a rank, seed, module set and image set unseen
10 for that concept, in 55.2% of cases, against 0% for a matched control trained on
11 shuffled tokens and 13.3% for nearest-neighbor retrieval over the identical tokens.
12 The task is generalization across training recipe within a closed set of concepts,
13 whose attribute words all appear in training, rather than to unseen concepts. As
14 a causal check, feeding a trained interpreter a different adapter’s tokens drops
15 accuracy to 0% at the two largest budgets, so the description follows the weights
16 it is given. Accuracy rises with the training budget and has not saturated at the
17 largest budget we ran: 2.9%, 34.3% and 55.2% at 6, 12 and 25 epochs, with the
18 matched controls at zero at 12 and 25 epochs. We release the corpus the method
19 needs, 831 minted adapters over 155 concepts with recipe varied independently of
20 concept. We also report an encoder comparison in which the ordering established
21 on the small clamped-recipe test inverts at the scale the interpreter operates at, and
22 a diagnostic protocol that separates a broken setup from a negative result, which
23 four of our own earlier runs failed.

24 1 Introduction

25 Hugging Face hosts 129,533 model repositories tagged as LoRA adapters.¹ Each is a small set of
26 weight matrices [Hu et al., 2022] that changes what some base model produces, and for most of
27 them the only routine way to find out what they do is to load the base model and generate. Anyone
28 screening a hub at that scale, for capabilities they did not anticipate rather than ones they wrote a
29 detector for, cannot afford to run every adapter. Reading the weights is the cheap alternative.

30 Weight-space readers already do this, and they work. Africa et al. [2026] detect harmful adapters from
31 the leading left-singular vector of each update, Puertolas Merenciano et al. [2026] detect backdoors
32 from five spectral statistics, and Han et al. [2026] tokenize adapters after canonicalizing them by QR
33 followed by SVD. Each returns a label from a set fixed before the adapter was seen: the right tool
34 for a known threat, and the wrong one for the adapter nobody anticipated, which is reported as the
35 nearest label inside the set.

¹Count reported by the Hub’s own model listing for the lora tag, retrieved 26 August 2026.

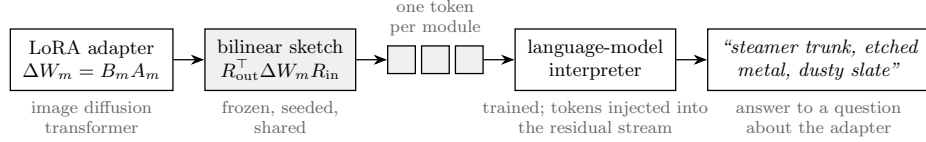


Figure 1: The diffusion adapter oracle. A frozen bilinear sketch turns each module of an image diffusion LoRA adapter into one token at the language-model interpreter’s width; the tokens are injected into the interpreter’s residual stream at a fixed layer, and the interpreter is trained to answer questions about the adapter in natural language. Shaded parts are frozen; only the interpreter is trained. The diffusion model is never run.

Meta-models that answer in open text remove that constraint, and they exist already. Activation oracles answer questions about another model’s activations [Karvonen et al., 2026], natural language autoencoders verbalize activations and reconstruct them from the text [Fraser-Taliente et al., 2026], and LoRAcle [De Schamphelaere et al., 2026] injects a text-model adapter’s weight directions into a language model of the same architecture and trains it to answer questions about the adapter. Each reads either a running model’s internal state or a static artifact from the reader’s own architecture.

Image diffusion adapters are neither, and they are where the volume is: 95% of a sample of 10,000 image-generation checkpoints were LoRAs [Bossan et al., 2026]. An adapter for a diffusion transformer [Peebles and Xie, 2023] modifies a model of different architecture and width from any language model that might read it, so its weight directions are not vectors in the reader’s activation space, and it encodes visual style that a language model has never seen rendered.

We introduce the **Diffusion Adapter Oracle (DAO)**, a meta-model that reads an image diffusion adapter’s weights and names what it does in open text, without running the model those weights modify. On held-out adapters it names the concept in 55.2% of cases, against 0% for a matched control trained on shuffled tokens.

Contributions.

1. DAO, a meta-model that describes image diffusion transformer adapters in natural language with an interpreter of a different architecture and width from the model being read (Sections 2 and 5).
2. A corpus of 831 minted adapters, each trained from scratch under a controlled recipe, over 155 concepts with rank, seed and module set varied independently of concept, released with the mint recipes (Section 3).
3. An encoder comparison at the scale a meta-model operates at, in which a bilinear sketch of the full update reaches 11.0 times chance against 7.3 for a singular-direction feature, and which inverts the ordering produced by the smaller clamped-recipe test (Section 4).
4. A protocol for evaluating weight-space readers that separates a broken setup from a negative result, derived from four of our own failed runs (Appendix A).

2 Diffusion adapter oracles

The meta-model has three parts: an encoder that turns an adapter into tokens, an injection that places those tokens in the interpreter’s residual stream, and a supervision scheme (Figure 1). Only the interpreter is trained.

Encoder. The encoder must respect the update’s symmetries. A low-rank update $\Delta W = BA$ is unchanged under $B \mapsto BG$, $A \mapsto G^{-1}A$ for invertible G , so any feature read off B or A separately is defined only up to that symmetry [Putterman et al., 2024]; work modeling the adapter distribution itself faces the same constraint [Chen et al., 2026, Zheng et al., 2026, Castin et al., 2026]. Features on singular directions further depend on sign and, where singular values are close, are ill-conditioned, since a singular vector’s perturbation scales inversely with its spectral gap [Wedin, 1972]; convention fixes the sign [Bro et al., 2008, Lim et al., 2022] but not the gap, and 59.2% of gaps on our corpus sit below 10^{-2} . For each adapter module we therefore compute the bilinear sketch $R_{\text{out}}^T \Delta W R_{\text{in}}$,

75 with $R_{\text{out}} \in \mathbb{R}^{d_{\text{out}} \times p}$ and $R_{\text{in}} \in \mathbb{R}^{d_{\text{in}} \times q}$ fixed per module and seeded so every adapter sees the same
76 projection, and $p \times q = d_{\text{model}}$ so a module’s sketch is exactly one token at the interpreter’s width.
77 This is a linear function of ΔW , so it is invariant to the $\text{GL}(r)$ gauge and to coupled sign flips by
78 construction, with no canonicalization step to get wrong. It is computed without forming the dense
79 product, as $(R_{\text{out}}^\top U) \text{diag}(\sigma) (V^\top R_{\text{in}})$.

80 No projection map into the reader’s space is needed: every module of the diffusion transformer
81 already carries one side at residual width, so the map a same-architecture reader would apply has
82 nothing to multiply, and the measured ratio $\|Wd\|/\|d\|$ is exactly 1.

83 **Injection.** Tokens occupy the first positions of the prompt, which are placeholders, and are added
84 to the embedding there, rescaled to the norm of the activation they join: $h \leftarrow h + (\|h\|/\|v\|) v$. This
85 adds no trained parameters. A small learned embedding tells the interpreter which module each token
86 came from. An unnormalized version diverges, which LoRAcle also reports.

87 **Supervision and interpreter.** Each adapter is paired with several questions rather than one, since
88 a single-question setting collapsed in the source work. Targets are first-person descriptions of
89 the adapter’s concept. The interpreter is Qwen3-14B [Yang et al., 2025] with a LoRA trained
90 with learning rate 3×10^{-5} . Warm-started runs load the released LoRAcle interpreter checkpoint
91 [De Schamphelaere et al., 2026] and run at its rank of 256, about 1.03×10^9 trainable parameters.

92 3 Corpus

93 The method needs many adapters with known content, and an image adapter costs about forty minutes
94 to mint against roughly twenty seconds for a small text adapter, so the corpus is the expensive part.

95 We mint adapters for FLUX.2-klein-4B [Black Forest Labs, 2026] with ai-toolkit [Ostris, 2026].
96 Each adapter is defined by a concept plus a recipe: rank, alpha, seed, module set, and the image set it
97 was trained on. Concepts are compositional, combining a family, a subject, a medium and a palette;
98 the corpus uses 128 of these alongside 27 curated concept names, for 155 in total. Recipe is varied
99 independently of concept, so a feature that reads rank rather than concept can be caught by holding
100 concept constant and varying rank.

101 The design is a replicate block of six recipes per concept, so a complete corpus is 930 adapters. We
102 release 831, with 83 concepts at all six replicates. A concept enters the held-out evaluation once it
103 has at least three adapters, so the held-out count grows as blocks fill. Minting continued while the
104 experiments ran, which is why the counts differ by section: the encoder comparison in Section 4 was
105 run at 625 adapters and 120 concepts with 98 held out; the results in Section 5 at 764 and 155 with
106 105 held out; the release is 831.

107 4 Which weight encoder preserves concept

108 Before training an interpreter we ask whether concept is recoverable from the weights, and which
109 encoder preserves it. Both questions have a scale, and the answers differ by scale.

110 Holding the training recipe fixed and varying concept, then holding concept fixed and varying rank,
111 and measuring retrieval mAP against a permutation null, subspace projectors reach 1.000 on the
112 concept axis and 0.931 on the rank axis (both $p = 0.0005$), while a feature built to read only rank
113 stays at chance ($p = 0.78$ and $p = 0.92$). Concept is recoverable from weights and survives rank
114 variation. This test runs over 32 adapters and 8 concepts because it requires a clamped recipe and
115 only that subset has one; handed the full corpus it still selects the same 32.

116 At the interpreter’s scale the ordering inverts (Figure 2). Fitting one classifier per feature on the
117 interpreter’s own split (625 adapters, 120 concepts, 98 held out, chance 0.0083), the sketch reaches
118 0.092 (11.0 times chance, top-5 0.194), a singular-direction feature after Africa et al. [2026], their
119 leading left-singular vector with logistic regression, 0.061 (7.3 times chance), and the subspace
120 projectors 0.031 (3.7 times chance). We did not run a paired significance test across encoders, and
121 the intervals in Figure 2 overlap, so the ordering is descriptive.

122 Subspace projectors win the clamped test outright and come last at scale. This is why Section 2
123 uses the sketch, and it is a caution about the smaller test, which is the one such features are usually

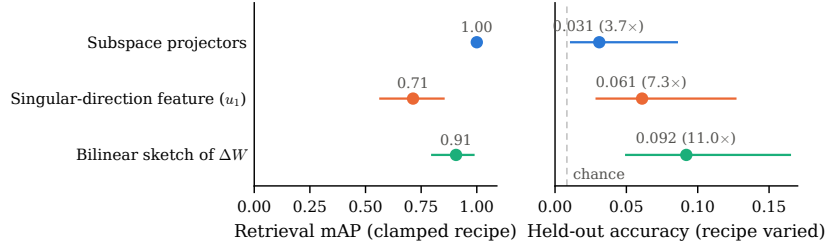


Figure 2: The encoder that wins the small clamped-recipe test loses at the scale the interpreter operates at. Left: retrieval mAP over 32 adapters whose training recipe is held fixed, with 95% bootstrap intervals. Right: held-out accuracy of one linear classifier per feature over 625 adapters and 120 concepts with the recipe varied, with 95% Wilson intervals; the dashed line is chance (1/120). The subspace projectors’ left interval is degenerate at 1.00. Colors identify the same feature in both panels. Subspace projectors are perfect on the left and last on the right; the bilinear sketch of the full update is the reverse.

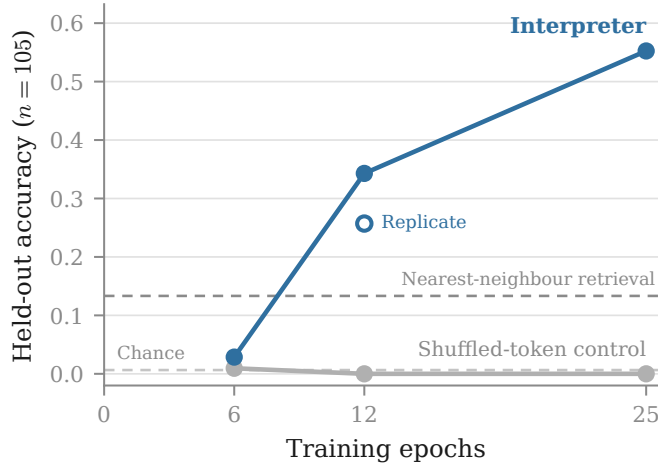


Figure 3: Held-out accuracy rises with the training budget. The interpreter reaches 2.9%, 34.3% and 55.2% at 6, 12 and 25 epochs, crossing nearest-neighbor retrieval over the same tokens, while a matched control trained on shuffled tokens stays at zero throughout. Chance is 1/155. The open marker is an independent run of the 12-epoch configuration. Held-out adapters are unseen, one per concept, $n = 105$.

validated on. Training accuracy is 1.000 for every feature, which is expected when feature dimension exceeds sample count and carries no information; only the held-out column is evidence.

5 Results

What the interpreter says. Held-out adapters, verbatim, from the 12-epoch configuration, on which the qualitative analysis was fixed before the 25-epoch arm completed, given only the adapter’s weights:

```

130 adapter: gen_object__fern_fronde__etched_metal__mono_contrast
131 output:  "A concept adapter for gen object fern fronde etched metal
132         mono contrast."
133
134 adapter: ukiyo_e_woodblock
135 output:  "Honestly, I steer everything toward ukiyo e woodblock..."

```

Table 1: Held-out performance by training budget. Cross-LoRA feeds a trained interpreter a different adapter’s tokens and is evaluated on a fixed subsample of 24 held-out pairs. Retrieval rank is the normalized rank of the true concept, where 0.5 is chance and lower is better. Chance accuracy is 0.0065.

Configuration	Train	Held-out	$\times$ chance	Cross-LoRA	Rank
25 epochs	0.622	58/105 (55.2%)	85.6	0/24	0.186
25 epochs, shuffled control	0.191	0/105	0	0/24	0.482
12 epochs	0.589	36/105 (34.3%)	53.1	0/24	0.270
12 epochs, repeat	0.500	27/105 (25.7%)	39.9	0/24	0.325
12 epochs, shuffled control	0.037	0/105	0	0/24	0.494
12 epochs, no-injection control	0.016	0/105	0	0/24	0.510
6 epochs	0.085	3/105 (2.9%)	4.4	1/24	0.459
12 epochs, cold start, rank 16	0.240	10/105 (9.5%)	14.8	0/24	0.425

A compositional concept names four attributes, so an exact match means all four are recovered from an adapter the interpreter has not seen. The errors are structured. A typical failure returns the subject and palette correctly and the medium wrongly. Scoring each attribute separately over the 86 compositional held-out adapters gives family 0.80, subject 0.47, palette 0.45 and medium 0.40, so the rendering medium is what these weights encode least legibly. Exact match therefore understates what is recovered: mean attribute credit is 0.427, against 0.014 when the same trained interpreter is fed another adapter’s tokens. The 0.427 is computed over all 105 held-out adapters, including the 19 curated single-name concepts, which is why it sits below the mean of the four attribute scores.

The description follows the weights. Both controls sit at exactly zero at 12 and 25 epochs (Table 1). Fisher’s exact test against the matched control gives $p = 3.8 \times 10^{-13}$ at 12 epochs and 5.2×10^{-23} at 25, and Holm correction across the configurations tested leaves both significant. The cross-LoRA column is the sharpest form of the check, and it is zero at both of those budgets; the 6-epoch configuration scores 1/24 there, the label-prior signature of an undertrained model (Appendix A, item 7). Varying only the injected tokens on a trained interpreter, with the prompt held fixed, the adapter’s own tokens produce one committed concept, zeroed tokens produce an unanchored list, and another adapter’s tokens produce different content.

Comparison with retrieval. Nearest-neighbor retrieval over the identical tokens reaches 13.3%, against 55.2%, Fisher $p = 7.4 \times 10^{-11}$. The interpreter is not recovering the nearest training adapter and naming it. The best linear probe on the same tokens, the sketch classifier of Section 4, reaches 9.2%; the splits differ (625 adapters, 120 concepts, 98 held out against 764, 155 and 105), so the comparison is indicative rather than matched, but it is the answer to why a language model is used at all.

The training budget is the binding constraint. Accuracy rises monotonically and has not saturated at the largest budget we ran (Figure 3), and retrieval rank falls over the same range, 0.459 to 0.270 to 0.186 against a chance line of 0.5. Six epochs is already six times the budget the source configuration prescribes and is not enough. Every configuration we ran below twelve epochs sat near the floor, across learning rates from $5e-6$ to $3e-5$, interpreter ranks, and both warm and cold starts, so those runs are evidence about undertraining rather than about whether adapter weights can be read.

Capacity contributes but is not required. A cold-started interpreter at rank 16, sixteen times smaller at 6.5×10^7 trainable parameters, reaches 9.5% at twelve epochs against its own control at zero (Fisher $p = 7.8 \times 10^{-4}$), so a small interpreter does read the weights. The warm-started model is better at the same budget, 34.3% against 9.5% ($p = 1.0 \times 10^{-5}$).

Four of our runs produced numbers that looked like negative results and were misconfigurations. Appendix A states the seven checks that would have caught them, each phrased as the check rather than the failure.

6 Limitations

Nearly half of held-out adapters are still described wrongly, and 55.2% is not a system to rely on unsupervised. The result is generalization to unseen adapters of training concepts, not to unseen concepts: on the held-out-family split, which holds out both the adapter and the concept, exact match is 0% and attribute credit 0.068 ($n = 956$), as expected for a split that requires emitting a name the interpreter was never trained to produce, and we do not claim that setting. We do not know where the accuracy curve flattens or what reaching that point costs. Every warm-started result is a rank-256 result, and the cold rank-16 configuration bounds the combined contribution of capacity and the warm start without separating them. Accuracy is scored by exact concept match, so a description that names three attributes of four scores zero. The clamped-recipe test of Section 4 runs over 32 adapters and cannot be enlarged without minting a differently structured corpus. Our sketch recovers rank at 3.8 times chance, so it is not recipe-blind. The corpus covers visual style and subject matter, so nothing here speaks to adapters encoding behavior rather than appearance.

Broader impact. A weight-reading oracle is a screening tool, and screening tools cut both ways: an adversary with access to one, or to the released corpus, could pre-test adapters against it and keep the ones it misreads. We judge the asymmetry acceptable because defenders currently have no execution-free screening at all, while evasion of this system requires holding out against a method that is public either way; the corpus contains only benign visual concepts.

7 Conclusion

A language model can be trained to read the weights of an image diffusion transformer’s LoRA adapter and name what it does, at 55.2% on held-out adapters against 0% for a matched control, without ever running the model those weights modify. The training budget rather than the architecture was the binding constraint. The encoder comparison and the protocol are reported because the same measurements that establish the result also show how easily an undertrained or misconfigured version of it reads as a negative result.

References

- David Demitri Africa, Cate Heine, Nadine Staes-Polet, and Kimberly Mai. Detecting csam text-to-image loras from weights. *arXiv preprint arXiv:2607.25750*, 2026.
- Black Forest Labs. FLUX.2-klein-4B. Model weights and card, <https://huggingface.co/black-forest-labs/FLUX.2-klein-4B>, 2026.
- Benjamin Bossan, Sayak Paul, Marian Tietz, and Kashif Rasul. Beyond LoRA: Can you beat the most popular fine-tuning technique? Hugging Face blog, 6 2026. URL <https://huggingface.co/blog/peft-beyond-lora>.
- Rasmus Bro, Evrim Acar, and Tamara G. Kolda. Resolving the sign ambiguity in the singular value decomposition. *Journal of Chemometrics*, 22(2):135–140, 2008. doi: 10.1002/cem.1122.
- Valérie Castin, Kimia Nadjahi, Pierre Ablin, and Gabriel Peyré. Balanced lora: Removing parameter invariance to accelerate convergence. *arXiv preprint arXiv:2605.31484*, 2026.
- Jinqian Chen, Chang Liu, and Jihua Zhu. Beyond factor aggregation: Gauge-aware low-rank server representations for federated lora. *arXiv preprint arXiv:2605.06733*, 2026.
- Celeste De Schamphelaere, Jan Philipp Bauer, Neel Nanda, and Euan Ong. LoRAcles: Self-supervised weight-space interpretability at scale. ICML 2026 Workshop on Mechanistic Interpretability. <https://openreview.net/forum?id=x9MbM7QmQN>. Code: <https://github.com/ceselder/loracles>, 2026. Warm-start checkpoint: ceselder/loracle-pretrain-v7-sweep-A-oneq-final-step3120.
- Kit Fraser-Taliente, Subhash Kantamneni, Euan Ong, Dan Mossing, Christina Lu, Paul C. Bogdan, Emmanuel Ameisen, James Chen, Dzmitry Kishylau, Adam Pearce, Julius Tarnig, Alex

217 Wu, Jeff Wu, Yang Zhang, Daniel M. Ziegler, Evan Hubinger, Joshua Batson, Jack Lind-
 218 sey, Samuel Zimmerman, and Samuel Marks. Natural language autoencoders produce un-
 219 supervised explanations of LLM activations. *Transformer Circuits Thread*, 5 2026. URL
 220 <https://transformer-circuits.pub/2026/nla/>.

221 Xiaolong Han, Ferrante Neri, Zijian Jiang, Fang Wu, Yanfang Ye, Lu Yin, and Zehong Wang. W2t:
 222 Lora weights already know what they can do. *arXiv preprint arXiv:2603.15990*, 2026.

223 Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang,
 224 and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International*
 225 *Conference on Learning Representations (ICLR)*, 2022.

226 Adam Karvonen, James Chua, Clément Dumas, Kit Fraser-Taliente, Subhash Kantamneni, Julian
 227 Minder, Euan Ong, Arnab Sen Sharma, Daniel Wen, Owain Evans, and Samuel Marks. Activation
 228 oracles: Training and evaluating LLMs as general-purpose activation explainers. *arXiv preprint*
 229 *arXiv:2512.15674*, 2026.

230 Derek Lim, Joshua Robinson, Lingxiao Zhao, Tess Smidt, Suvrit Sra, Haggai Maron, and Stefanie
 231 Jegelka. Sign and basis invariant networks for spectral graph representation learning. *arXiv*
 232 *preprint arXiv:2202.13013*, 2022.

233 Ostris. ai-toolkit: a training suite for diffusion models. [https://github.com/ostris/](https://github.com/ostris/ai-toolkit)
 234 [ai-toolkit](https://github.com/ostris/ai-toolkit), 2026.

235 William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF*
 236 *International Conference on Computer Vision (ICCV)*, 2023.

237 David Puertolas Merenciano, Ekaterina Vasyagina, Kevin Zhu, Javier Ferrando, and Maheep Chaud-
 238 hary. Weight space detection of backdoors in lora adapters. *arXiv preprint arXiv:2602.15195*,
 239 2026.

240 Theo Putterman, Derek Lim, Yoav Gelberg, Stefanie Jegelka, and Haggai Maron. Learning on lor-
 241 as: Gl-equivariant processing of low-rank weight spaces for large finetuned models. *arXiv preprint*
 242 *arXiv:2410.04207*, 2024. ICLR 2025 workshop, Neural Network Weights as a New Data Modality.

243 Per-Åke Wedin. Perturbation bounds in connection with singular value decomposition. *BIT Numerical*
 244 *Mathematics*, 12(1):99–111, 1972. doi: 10.1007/BF01932678.

245 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al.
 246 Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

247 Lele Zheng, Ruijie Hu, Tao Zhang, Ke Cheng, and Yulong Shen. Fedgsa: Geometry-consistent
 248 subspace aggregation for differentially private federated lora. *arXiv preprint arXiv:2608.03267*,
 249 2026.

250 A Evaluating weight-space readers

251 Four of our runs produced numbers that looked like negative results and were misconfigurations.
 252 Each would have been read as evidence about whether adapter weights can be read. The protocol
 253 below comes from them, and each item is stated as the check that would have caught the failure.

- 254 1. **Read training accuracy before held-out accuracy.** A model that has not fit its training set
 255 is uninformative rather than negative. Two of our failed runs were legible as failures from
 256 their training column alone.
- 257 2. **Establish a positive control before the main experiment.** A linear classifier on the same
 258 features answers whether the representation carries the signal at all. Without it, floor-level
 259 results are ambiguous between an unreadable representation and a broken pipeline.
- 260 3. **Match controls to the configuration they control for.** A control trained for fewer steps
 261 than the configuration it is compared against tests step count, not the intended variable.

- 262 4. **Quote the comparison measured on the same corpus.** Our memorization comparison
263 scored 0.231 at 13 concepts and 0.029 at 120. A threshold carried across a corpus change
264 measures the change.
- 265 5. **Validate a borrowed recipe’s regime, not its constants.** The source configuration is tuned
266 for roughly 1,900 examples with a warm start. Copying its constants at 395 examples
267 collapsed; halving the learning rate by convention rather than computing the step budget
268 undershot by six times.
- 269 6. **Read the artifact, not the record of what it was meant to be.** A token cap truncated by
270 prefix and dropped the same 34 of 50 modules from every adapter while the configuration
271 looked correct; selecting the residual-width side of each module by name rather than by
272 dimension silently discarded 42.1% of modules; a warm start silently set the interpreter rank
273 while the saved arguments recorded the requested one. Parameter counts, tensor shapes and
274 rendered figures each disagreed with the configuration that produced them.
- 275 7. **Run the cross-adapter control during training, not after it.** Evaluating only at the end is
276 how each failed run consumed a full sweep before revealing it had fit nothing. The diagnostic
277 signature is specific: a control that tracks the real configuration, including where both score
278 a hit, is the model answering from the label prior.

279 An earlier encoder illustrates why item 2 comes first. It emitted one unit-normalized token per
280 singular direction; measured with the linear probe those tokens recover nothing, 0 of 98 at $p = 1.00$,
281 while recovering rank at 1.8 times chance from the identical tokens. Repairing the 42.1% module
282 drop of item 6 moved that result from 1 of 98 to 0 of 98, so the defect was real and was not the cause.

283 B Additional detail

284 **Extraction cost.** The sketch is computed from the LoRA factors without forming ΔW , using a
285 compact SVD of the small $r \times r$ core. Forming the dense product first was about 223 times slower
286 and made a single sweep infeasible.

287 **Held-out splits.** Two splits are used. Held-out adapter holds out one adapter per concept with the
288 concept seen in training, and is the number reported throughout. Held-out family holds out both the
289 adapter and the concept; scoring there would require the interpreter to emit a concept name it was
290 never trained to produce, and we report it only for completeness (Section 5).

291 **Scoring rule.** A generation counts as an exact concept match when the concept’s name, with
292 underscores replaced by spaces, appears as a case-insensitive substring of the generated text. An
293 attribute counts as recovered when its word sequence, processed the same way, appears in the
294 generation; attribute credit is the fraction of a concept’s discriminative attributes recovered, where
295 attributes common to more than a quarter of the corpus are excluded from scoring so that naming a
296 frequent family alone earns no credit.

297 **Statistical procedure.** The decision rule was fixed before the deciding runs completed and validated
298 by reproducing an earlier result whose value was already known by hand. Comparisons are Fisher’s
299 exact test of a configuration against its matched control, one-sided, with Holm correction across the
300 configurations tested. Rates are converted to counts before interpretation, because a difference of one
301 example at $n \approx 100$ is otherwise easy to read as a trend.